

Inheritance – Part 1 (K Swart)

Using classes allows us to combine data and operations (methods) on that data in a single unit, a process called **encapsulation**.

Through encapsulation, an object becomes a self-contained entity. ONLY its own operations (methods) can directly access/manipulate the data.

In addition to implementing encapsulation, classes have other capabilities - you can create new classes from existing classes – the most common way is through **inheritance**.

This important feature encourages code re-use and saves programmers an enormous amount of time.

Inheritance allows us to extend the definition of a class, without making any physical changes to the existing class. It allows us to create new classes from existing classes.

Inheritance can be viewed as an “**is-a**” relationship eg. every Student **is a** Person.

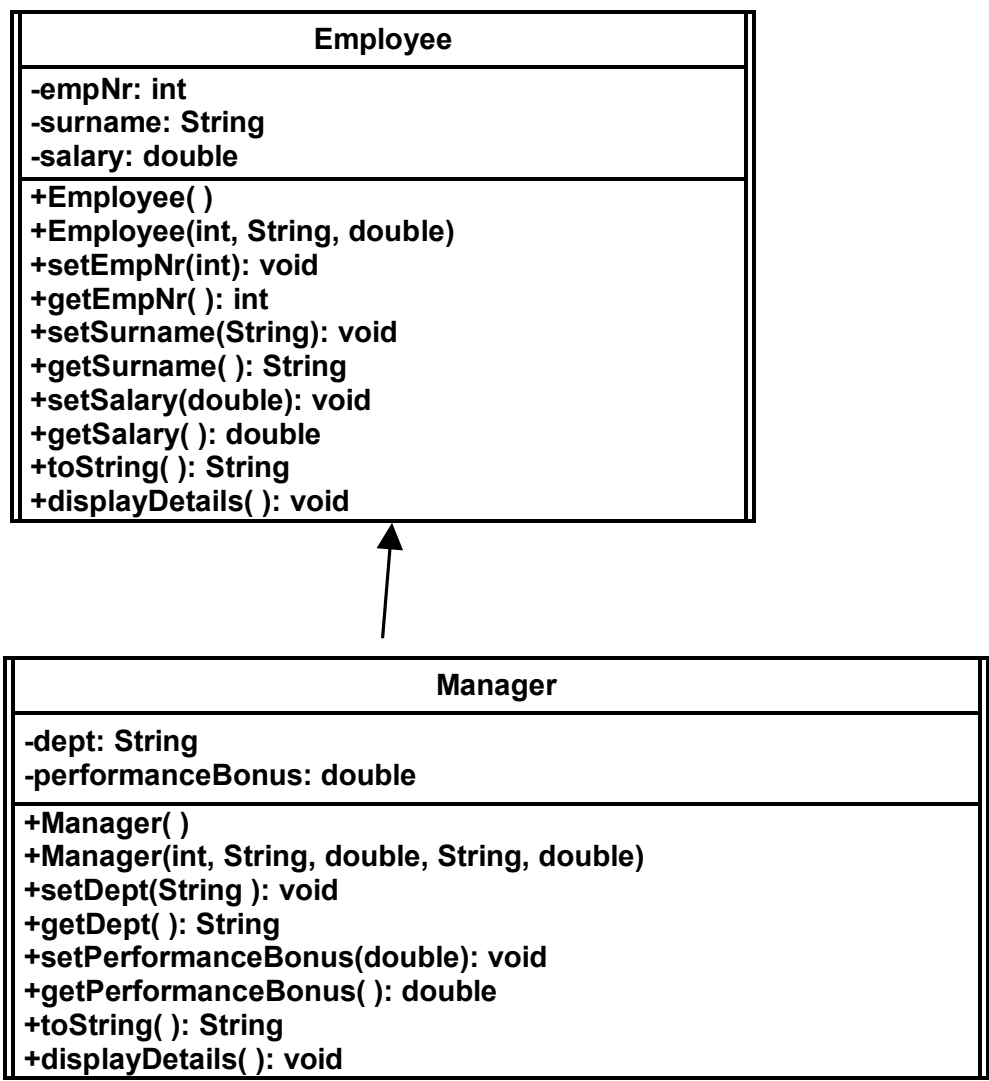
Any new class that you create from an existing class is called a **subclass** or **derived class**.

Existing classes are called **superclasses** or **base classes**.

The inheritance relationship enables a subclass to inherit features from its superclass.

Furthermore, the subclass can add new features of its own.

Consider the following UML diagram of the superclass, **Employee**, and subclass, **Manager**:



Consider the definition of the **Employee** class:

```
public class Employee {  
  
    private int empNr;  
    private String surname;  
    private double salary;  
  
    public Employee( ) {  
        empNr = 999999;  
        surname = "XXXXX";  
        salary = 0;  
    }  
  
    public Employee(int empNr, String surname, double salary){  
        this.empNr = empNr;  
        this.surname = surname;  
        this.salary = salary;  
    }  
}
```

setters and getters to follow.....

```
public void setEmpNr(int empNr) {  
    this.empNr = empNr;  
}
```

```
public int getEmpNr( ) {  
    return this.empNr;  
}
```

```
public void setSurname(String surname) {  
    this.surname = surname;  
}
```

```
public String getSurname( ) {  
    return this.surname;  
}
```

```
public void setSalary(double salary) {  
    this.salary = salary;  
}
```

```
public double getSalary( ) {  
    return this.salary;  
}
```

and finally, the **toString()** and **displayDetails()** methods.....

```
@Override
public String toString( ){
    String empString = "Emp#: " + this.empNr + ", Surname: " + this.surname
        + ", Salary: R" + salary;
    return empString;
}

public void displayDetails( ){
    System.out.println("Employee number: " + this.empNr);
    System.out.println("Surname: " + this.surname);
    System.out.println("Salary: R" + salary);
}

}
```

Definition of the class **Manager**, derived from the class **Employee**, with setters and getters:

```
public class Manager extends Employee {  
    private String dept;  
    private double performanceBonus;  
  
    public Manager( ){  
    }  
  
    public Manager(int empNr, String surname, double salary, String dept,  
                   double performanceBonus){  
    }  
  
    public void setDept(String dept){  
        this.dept = dept;  
    }  
  
    public String getDept( ){  
        return this.dept;  
    }  
  
    public void setPerformanceBonus(double performanceBonus){  
        this.performanceBonus = performanceBonus;  
    }  
  
    public double getPerformanceBonus( ){  
        return this.performanceBonus;  
    }  
}
```

and the **toString()** and **displayDetails()** methods.....

```
@Override
public String toString( ){
    String managerString = super.toString( ) + ", Manager of: " + this.dept
    + ", Performance bonus: R" + this.performanceBonus;
    return managerString;
}

@Override
public void displayDetails( ){
    super.displayDetails();
    System.out.println("Manager of: " + this.dept);
    System.out.println("Performance bonus: R" + this.performanceBonus);
    System.out.println("=====");
    System.out.println("Total salary: R" + (getSalary( )+performanceBonus));
}

}
```

In general, when writing the definitions of the methods of a subclass, to specify a call to a **public** method of a superclass, you do the following:

- if the subclass **OVERRIDES** a **public** method of the superclass, then you must specify a call to that **public** method of the superclass by using the reserved word **super**, followed by the dot operator, followed by the method name with the appropriate parameter list.
(See the call to the **displayDetails()** method of the **Employee** class, by a method of the **Manager** class on the previous page)
- if the subclass does **NOT** override a **public** method of the superclass, you can specify a call to that **public** method by using the name of the method and an appropriate parameter list.
(See the call to the **getSalary()** method of the **Employee** class, by a method of the **Manager** class on the previous page)